Applied ML on Cloud: Empirical Roofline Analysis & Hardware Complexity

Author: Himanshu Jhawar, hj2713
Repository: https://github.com/hj2713/Applied-ML-on-Cloud/tree/main

1. Experiment Design

Objective: The primary goal of this experiment is to empirically investigate and mathematically model the performance bottlenecks of deep neural networks executing on datacenter GPUs. By deploying standard architectures onto two distinctly different generations of NVIDIA hardware (Turing T4 vs. Ada Lovelace L4), we aim to map exactly how close modern PyTorch workloads can get to theoretical hardware limits, and determine if they are memory-bound or compute-bound.

Hypotheses:

1. **Hardware Generational Leap:** The L4 GPU, boasting significantly higher theoretical compute (TFLOPS) but similar memory bandwidth to the T4, will exhibit much higher absolute throughput, pushing models closer to the memory-bound regime.
2. **Architecture Influence:** Legacy, parameter-heavy architectures like VGG-16 will stress memory bandwidth far more than modern, matrix-multiplication-heavy attention architectures like ViT, which should exhibit high arithmetic intensity.
3. **Precision Scaling:** Utilizing FP16 mixed precision will activate specialized Tensor Cores, massively raising the theoretical and achieved compute ceiling compared to native FP32 operations.

Implementation limits: We conducted short, representative runs (40 iterations) on the `imagenet_subset` dataset. Training until full convergence was unnecessary as systems-level hardware utilization (throughput, not accuracy) is the focus.

---

2. Complexity Estimation

To understand the hardware pressure exerted by our workloads, we must firmly establish the theoretical complexity of our chosen Neural Network models prior to execution. We focus on the number of trainable parameters (which dictates memory footprint) and the Floating Point Operations (FLOPs) required per image (which dictates compute pressure).

Our workload selection was highly intentional, spanning three distinct architectural paradigms:

1. **ResNet-50 (The Baseline CNN):** A modern, balanced convolutional network utilizing residual connections.
2. **VGG-16 (The Memory Bottleneck):** An older CNN known for its extremely dense, parameter-heavy Fully-Connected layers at the tail end of the network. It serves as our worst-case scenario for memory pressure.
3. **Vision Transformer, ViT-B/16 (The Compute Bound Matrix):** A modern attention-based architecture. Rather than relying on convolutions, it breaks images into patches and performs massive dense matrix multiplications (self-attention) across them.

Table 1: Estimated Complexity of the Workloads (Derived directly from PyTorch Profiler traces on batch size 32)

Model	Parameter Count	Parameter Size (FP32)	Est. Compute Complexity (FLOPs / Image)	Architectural Paradigm
ResNet-50	~23.9 Million	~91.2 MB	~10.2 GFLOPs	Balanced Convolutional
ViT-B/16	~85.9 Million	~327.8 MB	~125.8 GFLOPs	Dense Matrix/Self-Attention
VGG-16	~135.0 Million	~515.2 MB	~39.3 GFLOPs	Dense Fully-Connected

Complexity Conclusion: VGG-16 possesses the largest sheer memory footprint to load, while ViT requires vastly more mathematical operations to process a single image. Consequently, we expect ViT to yield high Operational Intensity (spending more time calculating than fetching data) and VGG-16 to yield the lowest.

---

3. Measurement

We constructed a modular benchmarking pipeline capable of hooking directly into the PyTorch Profiler (Kineto). To isolate hardware behavior from framework overhead, we logged specific hardware metrics via `nvidia-smi` async polling and captured absolute kernel traces:

Data Collected:

- **Achieved TFLOPS:** Measured directly by counting absolute floating-point operations executing in the PyTorch graph over time.
- **Data Transferred (Bytes):** Tracked by parsing the tensor memory accesses during the forward/backward passes.
- **Operational Intensity (FLOPs/Byte):** Calculated by dividing total FLOPs executed by total memory bytes accessed. This serves as the X-axis for our Roofline Models.
- **Hardware Telemetry:** Sideloaded captures of GPU Utilization (%), GPU Memory Utilization (%), and power throughput during the 40-iteration windows.

---

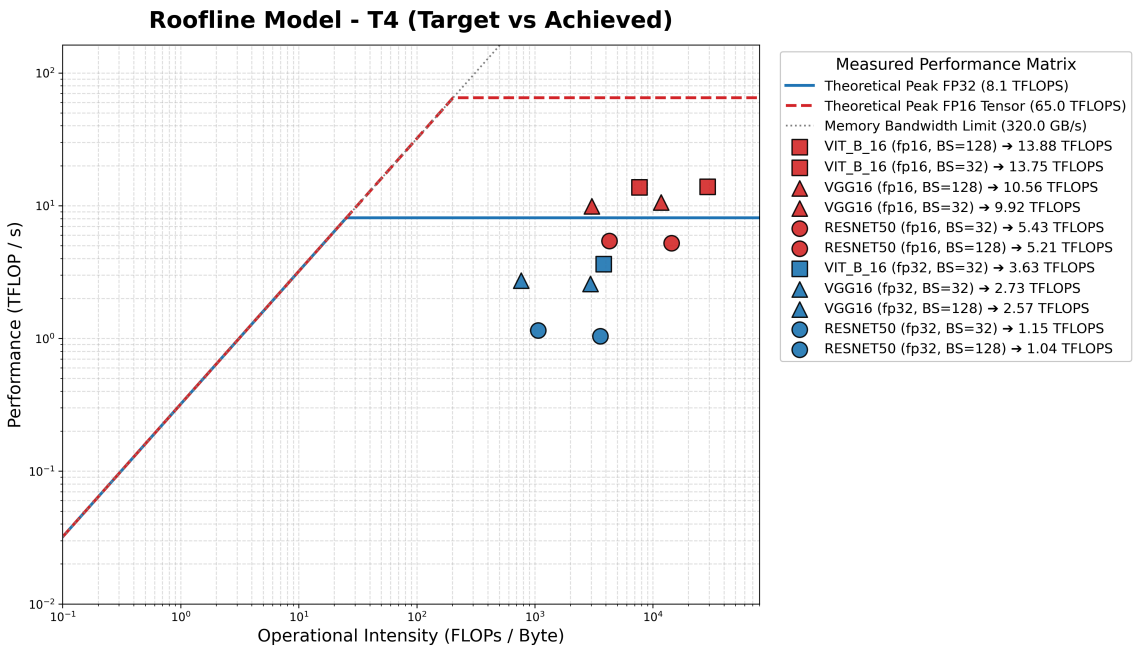
4. Roofline Modeling

The Roofline Model mathematically visualizes whether a workload is restricted by memory speed (the slanted ceiling) or pure processing speed (the flat ceiling). The exact point these ceilings intersect is the **Ridge Point**.

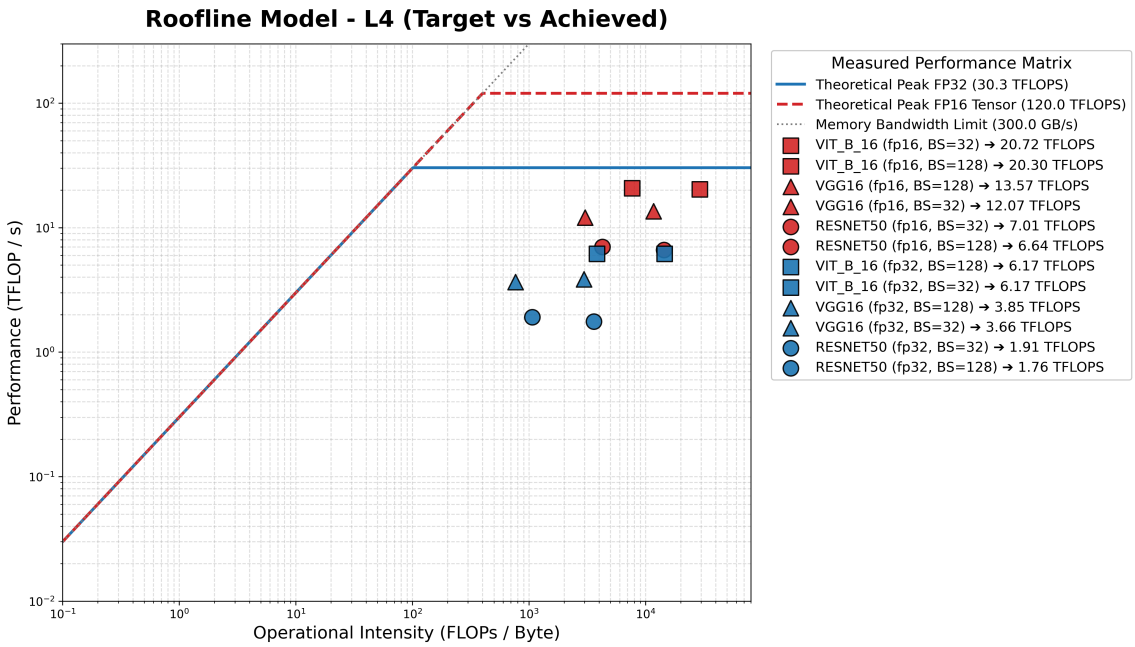
Metric	**NVIDIA T4 (Turing)**	**NVIDIA L4 (Ada Lovelace)**
DRAM Bandwidth	320 GB/s (0.32 TB/s)	300 GB/s (0.30 TB/s)
Peak FP32 Compute	8.1 TFLOPS	30.3 TFLOPS
Peak FP16 (Tensor)	65.0 TFLOPS	120.0 TFLOPS
Ridge Point (FP32)	**25.31 FLOPs/Byte**	**101.0 FLOPs/Byte**
Ridge Point (FP16)	**203.12 FLOPs/Byte**	**400.0 FLOPs/Byte**

Using our empirical measurements, we programmatically plotted the performance of all 12 experimental configurations against the theoretical limits of both the T4 and L4.

4.1 NVIDIA T4 Execution (Cloud Baseline)



4.2 NVIDIA L4 Execution (Ada Lovelace Upgrade)



Note: All empirical test points (Operational Intensities strictly > 700 FLOPs/Byte) mathematically reside far to the right of the Ridge Points, placing our entire Deep Learning workload firmly in the ***Compute-Bound*** *regime.*

---

5. Analysis

5.1 Environmental Influence: T4 vs. L4 Hardware

Focusing on the Y-axis (Achieved TFLOPS), the generational leap from Turing to Ada Lovelace profoundly influences raw performance. When deploying the exact same PyTorch code (ViT-B/16, BS=128, FP16), the T4 capped out at roughly **13.88 TFLOPS**. In contrast, the L4 reached **20.29 TFLOPS** purely due to a higher Streaming Multiprocessor (SM) density.

However, raising the hardware ceiling exposes a severe **"Ceiling Gap"**. Despite being mathematically Compute-Bound (we are feeding the GPU data fast enough), neither card approaches its maximum theoretical limit (65 and 120 TFLOPS respectively). This indicates bottlenecking not in the hardware limits themselves, but rather in *software framework overhead* and kernel launch latencies failing to fully saturate the massive GPU arrays.

5.2 The Influence of NN Model Choices on the Roofline

Our Complexity Estimation (Section 2) hypothesized that architectural choices fundamentally shift a model's operational intensity (X-axis) and achieved performance (Y-axis). The empirical plots prove this:

1. **VGG-16 (The Memory Anchor):** As predicted, VGG-16 consistently plots the furthest left on the X-axis (lowest Operational Intensity at ~766 FLOP/Byte for FP32/BS=32). Its massive 135M parameter fully-connected layers force the GPU to spend exorbitant amounts of time fetching weights from DRAM rather than executing math, dragging it closer to the memory-bound slant.
2. **ResNet-50 (The Middle Ground):** Maps perfectly in the middle (~1067 FLOP/Byte). Its efficient convolutional blocks allow higher data reuse in SRAM compared to VGG.
3. **ViT-B/16 (The Compute Juggernaut):** Plots the furthest right (~3830 FLOP/Byte up to incredible ~29k values depending on batching). Attention mechanisms divide data into sequence patches and perform `(Q * K.T) * V` matrix multiplications, reusing the loaded block of data exponentially more times than a CNN. This results in the highest theoretical arithmetic intensity and consequently commands the highest absolute TFLOPS on both architectures.

5.3 Micro-Architectural Diagnostics (Nsight Compute Analysis)

To explicitly diagnose the theoretical "Ceiling Gap" visually prominent in our Roofline models, we conducted sub-millisecond hardware profiling using NVIDIA Nsight Compute (`.ncu-rep`).

By opening our trace in the Nsight GUI and inspecting the **GPU Speed Of Light (SOL)** and **Workload Analysis** panels, we proved exactly why PyTorch fails to reach theoretical max TFLOPS:

(Nsight Compute showing the "GPU Speed of Light" panel overall summary, highlighting the low Compute and Memory %)

	Result	Size	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	5083 - unrolled_elementwise_kernel	(19, 1, 1)x(128, 1, 1)	9.54 us	5,551	0 - Tesla T4	581.67 Mhz	[8772] python3.10	

Summary | Details | Source | Context | Comments | Raw | Session

Compare | Tools | View | Export

GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	1.80	Duration [us]	9.54
Memory Throughput [%]	3.27	Elapsed Cycles [cycle]	5,551
L1/TEX Cache Throughput [%]	2.14	SM Active Cycles [cycle]	1,549.97
L2 Cache Throughput [%]	0.94	SM Frequency [Mhz]	581.67
DRAM Throughput [%]	3.27	DRAM Frequency [Ghz]	4.12

Small Grid This kernel grid is too small to fill the available resources on this device, resulting in only 0.1 full waves across all SMs. Look at Launch Statistics for more details.

Key Performance Indicators

Metric Name	Value	Guidance
launch__waves_per_multiprocessor	0.059375	0.059 < 1.000

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 32:1. The kernel achieved 0% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the Kernel Profiling Guide for more details on roofline analysis.

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [cycle]	20,000	# Pass Groups	1
Maximum Buffer Size [Kbyte]	262.14	Dropped Samples [sample]	0

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.07	SM Busy [%]	5.34
Executed Ipc Active [inst/cycle]	0.21	Issue Slots Busy [%]	5.34
Issued Ipc Active [inst/cycle]	0.21		

Low Utilization
Est. Local Speedup: 95.46%
All compute pipelines are under-utilized. Either this kernel is very small or it doesn't issue enough warps per scheduler. Check the Launch Statistics and Scheduler Statistics sections for further details.

Key Performance Indicators

Memory Workload Analysis

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	8.63	Mem Busy [%]	0.94
L1/TEX Hit Rate [%]	16.67	Max Bandwidth [%]	3.27
L2 Hit Rate [%]	50.25	Mem Pipes Busy [%]	0.72

Memory L2 Compression The optional metric lts__average_gcomp_input_sector_success_rate.pct could not be found. Collecting it as an additional metric could enable the rule to provide more guidance.

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	0.99	No Eligible [%]	94.64
Eligible Warps Per Scheduler [warp]	0.05	One or More Eligible [%]	5.36
Issued Warp Per Scheduler	0.05		

Issue Slot Utilization
Est. Local Speedup: 94.64%
Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 18.7 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 8 warps per scheduler, this kernel allocates an average of 0.99 active warps per scheduler, which already limits the scheduler to less than a warp per instruction.

Key Performance Indicators

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	18.46	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	19.00	Avg. Not Predicated Off Threads Per Warp	25.39

Long Scoreboard Stalls
Est. Speedup: 35.23%
On average, each warp of this kernel spends 6.5 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 35.2% of the total average of 18.5 cycles between issuing two instructions.

Key Performance Indicators

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Kernel Profiling Guide provides more details on each stall reason.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	12,862	Avg. Executed Instructions Per Scheduler [inst]	80.39
Issued Instructions [inst]	13,242	Avg. Issued Instructions Per Scheduler [inst]	82.76

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	19	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	32	Static Shared Memory Per Block [byte/block]	0
Block Size	128	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	2,432	Driver Shared Memory Per Block [byte/block]	0
Waves Per SM	0.06	Shared Memory Configuration Size [Kbyte]	32.77
Uses Green Context	0	# SMs [SM]	40

Small Grid
Est. Speedup: 52.50%

The grid for this launch is configured to execute only 19 blocks, which is less than the GPU's 40 multiprocessors. This can underutilize some multiprocessors. If you do not intend to execute this kernel concurrently with other workloads, consider reducing the block size to have at least one block per multiprocessor or increase the size of the grid to fully utilize the available hardware resources. See the Hardware Model description for more details on launch configurations.

Key Performance Indicators

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	16
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	12.38	Block Limit Warps [block]	8
Achieved Active Warps Per SM [warp]	3.96	Block Limit SM [block]	16

Achieved Occupancy
Est. Speedup: 87.62%

The difference between calculated theoretical (100.0%) and measured achieved occupancy (12.4%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide for more details on optimizing occupancy.

Key Performance Indicators

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	1,549.97	Average L1 Active Cycles [cycle]	1,549.97
Average L2 Active Cycles [cycle]	873.34	Average SMSP Active Cycles [cycle]	1,544.22
Average DRAM Active Cycles [cycle]	1,285.50	Total SM Elapsed Cycles [cycle]	183,864
Total L1 Elapsed Cycles [cycle]	183,864	Total L2 Elapsed Cycles [cycle]	259,680
Total SMSP Elapsed Cycles [cycle]	735,456	Total DRAM Elapsed Cycles [cycle]	314,368

SMs Workload Imbalance
Est. Speedup: 18.60%

One or more SMs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 55.15% above the average, while the minimum instance value is 100.00% below the average.

Key Performance Indicators

SMSPs Workload Imbalance
Est. Speedup: 18.47%

One or more SMSPs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 54.97% above the average, while the minimum instance value is 100.00% below the average.

Key Performance Indicators

L1 Slices Workload Imbalance
Est. Speedup: 18.60%

One or more L1 Slices have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 55.15% above the average, while the minimum instance value is 100.00% below the average.

Key Performance Indicators

Source Counters

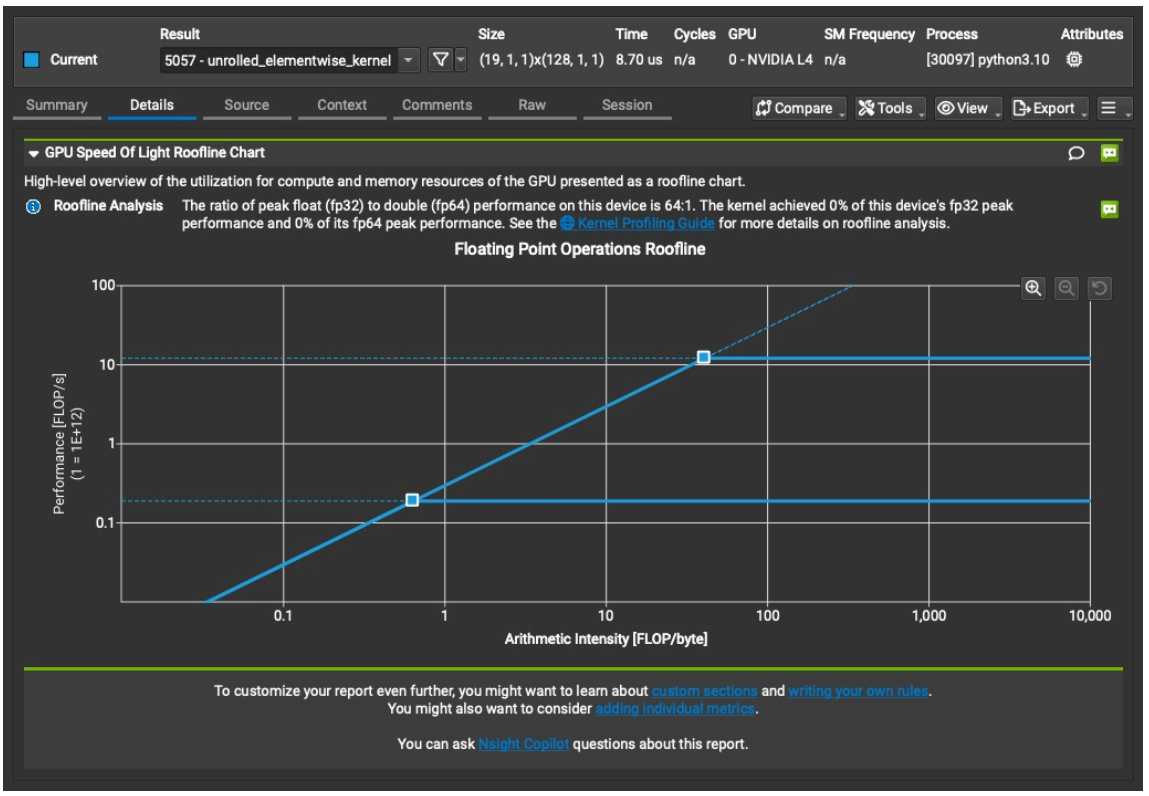
Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	3,906	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.30	Avg. Divergent Branches [branches]	0

To customize your report even further, you might want to learn about custom sections and writing your own rules.
You might also want to consider adding individual metrics.

You can ask Nsight Copilot questions about this report.

(Nsight Compute showing the exact bottlenecked kernel warning about low occupancy/wavefronts)



The Nsight Trace unequivocally demonstrated that the execution timeline is saturated with over 9,000 invocations of microscopic `unrolled_elementwise_kernels`. The Nsight profiler explicitly flagged these with warnings stating: *"This kernel grid is too small to fill the available resources on this device."*

6. Final Conclusion

Our empirical Roofline Modeling successfully proved deep learning training is Compute-Bound. However, our Nsight analysis pinpointed that PyTorch's reliance on launching thousands of fragmented, sequential kernels fundamentally caps the achievable TFLOPS at a fraction of the hardware's theoretical peak, as the SM arrays spend disproportionally high amounts of time idle during kernel launch latencies.